WEEK 5 - PYTHON PROGRAMMING

# pandas

Series, DataFrames and a week of stock trading

Session 2 - 120 minutes, including a break

Instructor: MinGi KYUNG

# Learning Goals & Agenda

- Meet Series and DataFrames, labeled arrays.
- Track one stock, then a whole portfolio.
- Select, filter and sort table data.
- Summarize columns with statistics.

Review 10 min / pip & Series 25 min / Frames & selection 15 min / Break 10 min

Sorting & stats 15 min / Columns & counting 15 min / Guided labs 25 min / Review & quiz 5 min

# Review: From NumPy to pandas

- Yesterday: NumPy arrays, vector math, masks.
- Arrays answer by position: row 2, column 0.
- Real data answers by name: CODE, Tue, close.
- pandas adds labels and tables on arrays.

Today's story: your first week of stock trading.

guessing.py is due now.

```
import pandas as pd

closes = pd.Series([10000, 10200, 9800])
print(closes.values)   # a NumPy array inside
```

# Why pandas Exists

- Yesterday's arrays hold plain numbers.
- A portfolio mixes tickers, shares, prices.
- Each column needs a name; rows, labels.
- pandas is the standard tool for that data.

```
Question      NumPy       pandas
same math     yes         yes, arrays inside
mixed types   one dtype   a type per column
row names     0, 1, 2     any labels you like
```

# Installing pandas: pip

- pandas is installed exactly like NumPy.
- Locally: pip install pandas, in the terminal.
- Colab ships pandas already.
- The convention: import pandas as pd.

```
$ pip install pandas
Successfully installed pandas-2.2.3 ...

import pandas as pd   # pd, like np
```

# Series: A Labeled Column

- A Series is one column of values.
- pd.Series([...]) builds it from a list.
- Here: CODE's closing prices, in won.
- Math works on the whole column at once.

```
import pandas as pd

closes = pd.Series([10000, 10200, 9800])
print(closes)
print(closes.tolist())   # [10000, ...]
```

# Vector Math, Zero Loops

- closes - 10000: every day's gain at once.
- You bought CODE at 10,000 on Monday.
- mean, max, min, sum aggregate at once.
- Summing a mask counts the winning days.

```
import pandas as pd

closes = pd.Series([10000, 10200, 9800,
                    10500, 10800])
print((closes - 10000).tolist())  # [0, 200,...]
print(closes.mean())              # 10260.0
print(closes.max())               # 10800
print((closes > 10000).sum())     # 3
```

# The Index: Names for Rows

- The index labels each row.
- Default labels are positions: 0, 1, 2, 3.
- index= gives day names, Mon to Fri.
- Square brackets and .loc read by label.

```
import pandas as pd

closes = pd.Series([10000, 10200, 9800,
                    10500, 10800],
                   index=["Mon", "Tue",
                          "Wed", "Thu", "Fri"])
print(closes["Tue"])       # 10200
print(closes.loc["Mon"])   # 10000
```

# Masks: Keep What You Want

- A comparison returns True/False per day.
- Masks work exactly as in NumPy.
- closes[mask] keeps only winning days.
- Combining tests needs parentheses and &.

```
import pandas as pd

closes = pd.Series([10000, 10200, 9800,
                    10500, 10800])
mask = closes > 10000

print(closes[mask].tolist())   # [10200, ...]
print(closes[mask].mean())     # 10500.0
print(mask.tolist())           # False, True
```

# Practice 1: One Stock, One Week

- Open first_stock.py.
- The starter holds CODE's Mon-Fri closes.
- Print the mean rounded, the high, the count.
- Print each winning day with its close. 5 minutes.

You bought at 10,000; winning days close higher.

```
Mean: 10260.0
High: 10800
Above buy (10000): 3
Tue 10200
Thu 10500
Fri 10800
```

# DataFrame: A Table of Columns

- A DataFrame is a table: rows by columns.
- Here: your Friday-evening portfolio.
- A dict of equal-length lists builds one.
- Each column is a Series sharing one index.

CODE is CodeShop, yesterday's store.

```
import pandas as pd

portfolio = pd.DataFrame({
    "ticker": ["CODE", "TEA", "ROBO"],
    "shares": [10, 40, 5],
    "buy": [10000, 5000, 40000],
    "close": [10800, 4800, 45000],
})
print(portfolio)
```

# Two Ways to Build a Frame

- Dict of lists: one key per column.
- List of dicts: one dict per trade.
- Row dicts share the same keys.
- Both shapes build the same table.

```
import pandas as pd

trades = [
    {"ticker": "CODE", "shares": 10},
    {"ticker": "TEA", "shares": 40},
]
log = pd.DataFrame(trades)
print(log.shape)   # (2, 2)
```

# First Look: shape, head, columns

- df.shape is (rows, columns): here (3, 4).
- df.head(n) previews the first rows.
- df.tail(n) previews the last rows.
- df.columns lists the column names.

```
import pandas as pd

portfolio = pd.DataFrame({
    "ticker": ["CODE", "TEA", "ROBO"],
    "shares": [10, 40, 5],
    "buy": [10000, 5000, 40000],
    "close": [10800, 4800, 45000],
})

print(portfolio.shape)          # (3, 4)
print(portfolio.head(2))
print(portfolio.columns.tolist())
```

# Selecting Columns

- One column: portfolio["close"] gives a Series.
- Several: a list of names inside brackets.
- The list's order sets the column order.
- df.col works when the name is simple.

```
import pandas as pd

portfolio = pd.DataFrame({
    "ticker": ["CODE", "TEA", "ROBO"],
    "shares": [10, 40, 5],
    "buy": [10000, 5000, 40000],
    "close": [10800, 4800, 45000],
})

tickers = portfolio["ticker"]
print(tickers.tolist())   # ['CODE', ...]

friday = portfolio[["ticker", "close"]]
print(friday.shape)       # (3, 2)
```

# Rows by Position: iloc

- iloc selects rows by position.
- portfolio.iloc[0] is your first trade.
- Slices keep several rows: iloc[0:2].
- Negative positions count from the end.

```
import pandas as pd

portfolio = pd.DataFrame({
    "ticker": ["CODE", "TEA", "ROBO"],
    "shares": [10, 40, 5],
    "buy": [10000, 5000, 40000],
    "close": [10800, 4800, 45000],
})

print(portfolio.iloc[0])
print(portfolio.iloc[0:2])
print(portfolio.iloc[-1]["ticker"])   # ROBO
```

# Break & Midpoint Check

10-minute break

- Series or DataFrame: one column or a table?
- What exactly does a mask keep?
- Why must every column list have one length?

Next: sorting, statistics and your profit.

# Sorting: sort_values

- sort_values("close") sorts by one column.
- ascending=False puts the biggest first.
- Whole rows travel with their sort key.
- Sorting returns a new frame; yours is safe.

```
import pandas as pd

portfolio = pd.DataFrame({
    "ticker": ["CODE", "TEA", "ROBO"],
    "shares": [10, 40, 5],
    "buy": [10000, 5000, 40000],
    "close": [10800, 4800, 45000],
})

ranked = portfolio.sort_values("close",
                               ascending=False)
print(ranked["ticker"].tolist())
# ['ROBO', 'CODE', 'TEA']
```

# One-Column Statistics

- Column math from Series works here.
- describe() prints the whole summary.
- count, mean, std, min, max, quartiles.
- round(x, 2) keeps reports tidy.

```
import pandas as pd

portfolio = pd.DataFrame({
    "ticker": ["CODE", "TEA", "ROBO"],
    "shares": [10, 40, 5],
    "buy": [10000, 5000, 40000],
    "close": [10800, 4800, 45000],
})

closes = portfolio["close"]
print(closes.max())             # 45000
print(round(closes.mean(), 1))  # 20200.0
print(closes.describe())
```

# Where Is the Best Row? idxmax

- idxmax() returns the label of the biggest.
- argsort ranked by position; idxmax names it.
- .loc[label] reads the whole row.
- Works on a Series or on any column.

```
import pandas as pd

closes = pd.Series([10000, 10200, 9800,
                    10500, 10800],
                   index=["Mon", "Tue",
                          "Wed", "Thu", "Fri"])
print(closes.idxmax())          # Fri
print(closes[closes.idxmax()])  # 10800
```

# New Columns from Old

- Assignment adds a column to the frame.
- invested: what you paid; profit: close - buy.
- Vector math fills every row at once.
- The frame changes in place.

```
import pandas as pd

portfolio = pd.DataFrame({
    "ticker": ["CODE", "TEA", "ROBO"],
    "shares": [10, 40, 5],
    "buy": [10000, 5000, 40000],
    "close": [10800, 4800, 45000],
})

shares = portfolio["shares"]
buy = portfolio["buy"]
close = portfolio["close"]

portfolio["invested"] = shares * buy
portfolio["profit"] = (close - buy) * shares
print(portfolio["profit"].tolist())
# [8000, -8000, 25000]
```

# Counting Categories: value_counts

- value_counts() tallies each value.
- Counts arrive largest first.
- Your journal: buys versus sells.
- Works on strings, labels, any column.

```
import pandas as pd

actions = pd.Series(["buy", "sell", "buy",
                     "buy", "sell", "buy"])
print(actions.value_counts())
# buy: 4, sell: 2
```

# Filtering Whole Rows

- A mask on a column filters whole rows.
- close above buy keeps CODE and ROBO.
- Combine masks: (a) & (b), (a) | (b).
- Assign the result; the original is safe.

```
import pandas as pd

portfolio = pd.DataFrame({
    "ticker": ["CODE", "TEA", "ROBO"],
    "shares": [10, 40, 5],
    "buy": [10000, 5000, 40000],
    "close": [10800, 4800, 45000],
})

win = portfolio["close"] > portfolio["buy"]
winners = portfolio[win]

print(winners["ticker"].tolist())
# ['CODE', 'ROBO']
print(winners.shape)   # (2, 4)
```

# Practice 2: Rank Your Positions

- Open positions.py.
- Add profit = (close - buy) * shares.
- Print tickers and profits sorted, top first.
- Print the top ticker and the winner count. 5 minutes.

idxmax finds the row; the mask counts the rest.

```
['ROBO', 'MLBK', 'CODE', 'TEA']
[25000, 15000, 8000, -8000]
Top: ROBO
Winners: 3
```

# Lab 1: Portfolio Report

- Open my_portfolio.py.
- Print the frame's shape.
- Add value = shares * close.
- Print values, the top holding, the mean.

```
Shape: (5, 4)
Value: [108000, 192000, 225000, 75000, 84000]
Top holding: ROBO
Mean value: 136800.0
```

# Lab 1 Checks

- Shape reports (5, 4): five rows, four columns.
- value arrives as one vector product.
- idxmax plus .loc names ROBO.
- round(..., 1) keeps the mean tidy.

```
Friday evening, four answers
Shape        (5, 4), no counting by eye
Value        one column, no loop
Top holding  ROBO, 225000 won
Mean         684000 over 5 positions
```

# Lab 2: Trade Journal

- Open trade_log.py.
- Count actions with value_counts.
- Print action names and their counts.
- Print total traded and the big trades.

```
Actions: ['buy', 'sell']
Counts: [6, 4]
Traded: 340000
Big trades: ['buy', 'buy', 'buy', 'buy']
```

# Lab 2 Checks

- Counts arrive largest first: 6 buys, 4 sells.
- Traded sums the whole amount column.
- The mask keeps the four 40,000+ trades.
- Filtering never touches the original log.

```
Questions behind the numbers
More buys or sells?   buys, 6 of 10
Average trade         34000
Big trades            all four are buys
```

# Lab 3 (Take-Home): A Week in Review

- Create price_report.py.
- Read FIT's five closes, Mon to Fri.
- Build a Series with days as the index.
- Print mean, high, best day, above-mean count.

Assume integer-formatted input.

```
Close Mon: 4200
Close Tue: 4350
Close Wed: 4100
Close Thu: 4400
Close Fri: 4300
Mean: 4270.0
High: 4400
Best day: Thu 4400
Above mean: 3
```

# How the Week Report Works

- Read first, convert, then build the Series.
- days and closes share one order.
- mean, max, idxmax, mask: four answers.
- The index turns prices into a report.

Your program reads the five closes with input().

```
import pandas as pd

days = ["Mon", "Tue", "Wed", "Thu", "Fri"]
typed = ["4200", "4350", "4100", "4400",
         "4300"]

values = []
for text in typed:
    values.append(int(text))
closes = pd.Series(values, index=days)

print(closes.mean())      # 4270.0
print(closes.idxmax())    # Thu
```

# Common Mistakes

- pip install runs in the terminal, not in Python.
- DataFrame columns must be equal-length lists.
- One column gives a Series; two brackets a frame.
- Parenthesize each side when combining masks.

Before running: name the type you expect back.

When debugging: print df.shape first to see what you have.

# Exit Quiz & Submission

1. Which command installs pandas?
2. Which pandas type holds one labeled column?
3. What type is df["close"]?
4. Which call returns the label of the largest value?
5. What does df[df["profit"] > 0] give you?

Submit first_stock.py, positions.py, my_portfolio.py and trade_log.py. price_report.py is due next session. Record at least three test cases in comments for each program.

# Next Session: Files & Classes

- Read text files line by line with a for loop.
- Save results that outlive the program.
- Then design your own types with classes.

Preparation: keep my_portfolio.py handy; its table becomes a CSV file in Week 6.